DAYANANDA SAGAR ACADEMY OF TECHNOLOGY AND MANAGEMENT

(Affiliated to VTU, Belagavi and Approved by AICTE, New Delhi),
CE, CSE, ECE, EEE, ISE, ME Courses Accredited by NBA, New Delhi, NAAC A+

Intelligent Language Processing – BCS701

Project Report

"Conversational AI Agent for Commerce"

Submitted in partial fulfillment of the requirement for the award of the degree of

Bachelor of Engineering

in

Computer Science and Engineering

Submitted By

Aditya [1DT23CS007]
Aparajeeta Singh [1DT23CS024]
Chiranjeevi M G [1DT23CS045]

Under the Guidance of

Prof. Orchu Venkata Koteswara Rao
Assistant Professor, Department of CSE

Department of Computer Science and Engineering

DAYANANDA SAGAR ACADEMY OF TECHNOLOGY AND MANAGEMENT
(Affiliated to VTU, Belagavi and Approved by AICTE, New Delhi),
CE, CSE, ECE, EEE, ISE, ME Courses Accredited by NBA, New Delhi, NAAC A+

ACADEMIC YEAR 2026-27

Abstract

This project presents an autonomous, goal-driven conversational AI agent designed for **Intelligent Language Processing** applications. Moving beyond traditional rule-based chatbots and static text generators, the system integrates natural language intent resolution, Retrieval-Augmented Generation (RAG), structured function execution, and self-evaluating safety guardrails. The agent dynamically processes user queries, retrieves domain knowledge through dense vector similarity search across structured databases (catalog/catalog.json, catalog/vector_store.py), and executes end-to-end operational workflows such as cart modifications and order checkouts (payments/razorpay_client.py) while retaining multi-turn session context.

To ensure system reliability, safety, and robustness prior to deployment, the architecture features rate-limiting controls (gating/limits.py), real-time operational logging (audit/logger.py), and a multi-tiered evaluation framework. The agent was rigorously evaluated using offline ground-truth datasets (eval/offline_eval.py), automated adversarial red-teaming (red_team_tests.py), and an automated LLM-as-a-Judge pipeline (eval/llm_judge.py). Results confirm that the architecture achieves high retrieval precision, accurate structured tool invocation, robust prompt injection defense, and reliable adherence to operational guardrails

Introduction

In recent years, Natural Language Processing (NLP) has undergone a major shift from basic text classification and template-based response generation toward dynamic, autonomous conversational agents. Modern Intelligent Language Processing demands AI interfaces that go beyond answering simple static questions; today's applications require systems capable of understanding ambiguous user intent, accessing up-to-date domain data, and executing multi-step operational tasks across external software APIs and databases. Conversational agents now serve as intelligent, automated intermediaries between human users and complex enterprise infrastructure.

Building a truly intelligent agent, however, requires solving several fundamental challenges in language understanding, retrieval grounding, and decision-making. Standard Large Language Models (LLMs) excel at generating human-like text but operate on static pre-training weights. As a result, they cannot independently access real-time inventory assets or perform real-world actions without specialized architectural extensions. Conversely, legacy rule-based chatbots offer predictable control but lack the flexibility to parse non-standard user phrasing or handle complex multi-turn dialogue state transitions.

To bridge this gap, this project develops a production-ready, domain-aware conversational agent tailored for Intelligent Language Processing applications. Rather than acting merely as an open-ended conversational engine, this system functions as an autonomous decision-making pipeline. The agent is engineered to ingest natural language user queries, filter out malformed or malicious inputs, retrieve real-time domain context via dense vector embeddings, format schema-compliant tool calls for backend execution, and synthesize concise, factual natural language responses.

Problem Statement

Standard conversational interfaces and traditional rule-based chatbots suffer from critical design limitations that prevent them from handling real-world operational workflows:

1. **Context Loss & Static Knowledge:** Standard Large Language Models (LLMs) operate on static pre-training weights. They lack real-time access to operational assets, domain inventories, and structured enterprise catalogs, leading to information hallucination when queried about current domain data.

2. **Lack of Action & Tool Agency:** Most basic chatbots are limited to text generation and cannot invoke external functions, parse operational APIs, or manage stateful workflows such as cart modifications or transaction processing.

3. **Ambiguity & Intent Resolution Failures:** User inputs in real-world scenarios are frequently noisy, ambiguous, or multi-faceted. Traditional rule-based agents fail to resolve complex multi-turn intents, while unrestricted generative models risk executing incorrect or unintended backend operations.

4. **Safety, Guardrails, & Exploitation Risks:** Unbounded conversational agents are highly vulnerable to adversarial prompts, prompt injection attacks, rate abuse, and unauthorized boundary execution.

To address these core challenges, this project formulates a robust agentic system that unifies semantic vector retrieval (RAG), structured API tool execution, automated logging, and adversarial guardrails into a single intelligent language architecture.

System Architecture & Intelligent Agent Design

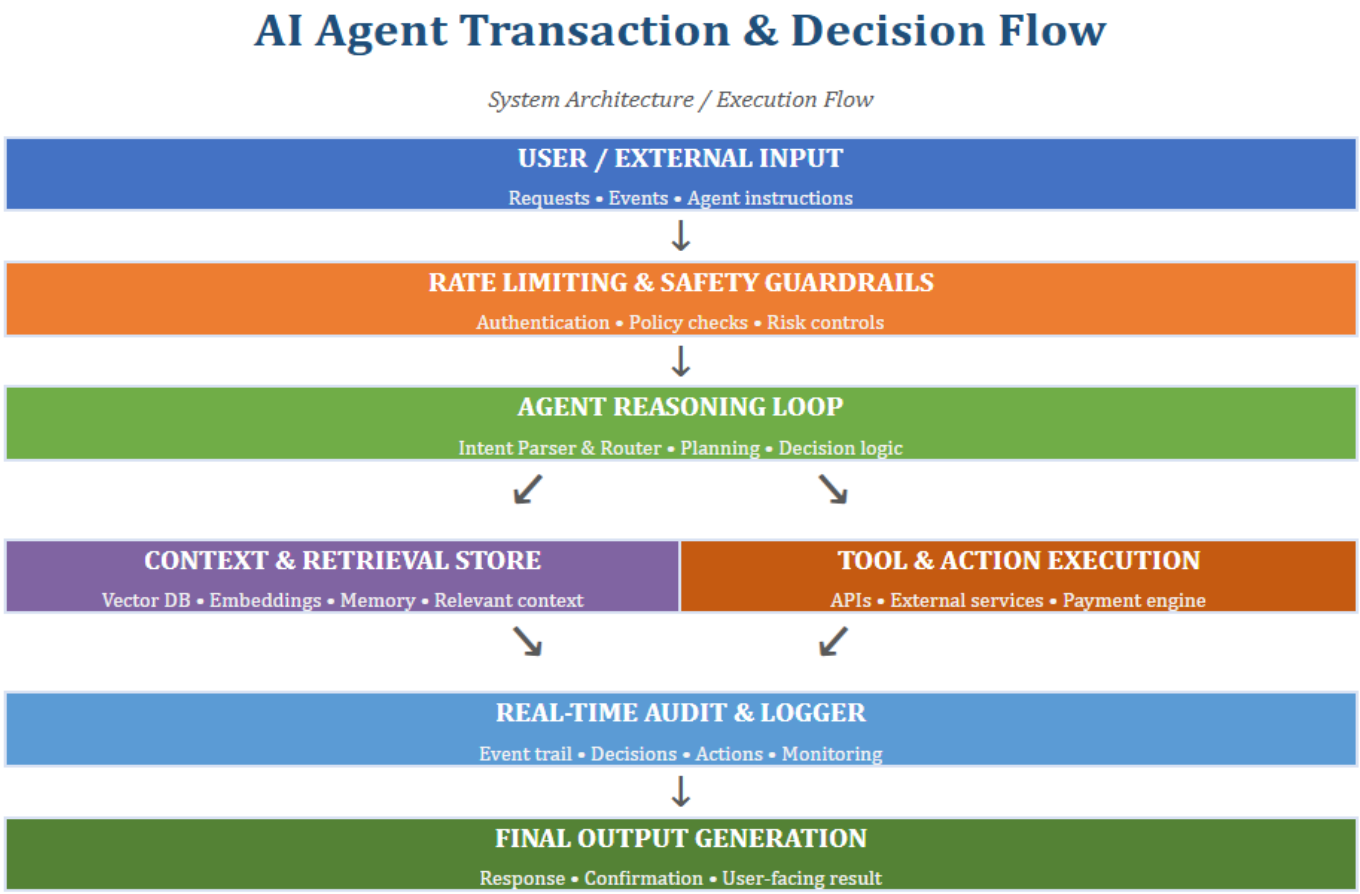


Figure 1. System Architecture

2.1 Agent Workflow & Reasoning Loop

The core engine follows a closed-loop perception-reasoning-action model:

1. **Input Ingestion & Filtering:** User prompts pass through input sanitization and safety gating layers.

2. **Intent Parsing & Semantic Routing:** The primary LLM analyzes the query to determine whether it requires direct textual generation, catalog search, or specific tool execution.

3. **Retrieval-Augmented Generation (RAG):** For product/knowledge queries, the agent computes vector similarity against indexed product catalogs (catalog.json / vector_store.py) to retrieve accurate context.

4. **Tool Invocation & State Tracking:** If the query requests action (e.g., cart modification, payment checkout via Razorpay), the agent formats structured tool calls and tracks state changes.

5. **Response Synthesis:** The agent aggregates tool outputs or retrieved context into a natural language response.

2.2 System Modules

- **Agent Core (agent/agent.py):** Orchestrates dialogue flow, tool calling, and multi-turn state.
- **Retrieval Unit (catalog/vector_store.py):** Handles embedding-based search for accurate domain context.
- **Audit & Analytics Engine (audit/logger.py, analytics.py):** Records interaction logs, tool calls, latency, and success metrics in real time.
- **Gating & Safety Enforcement (gating/limits.py):** Prevents abuse by enforcing system operational boundaries.

Implementation, Dataset & Agent Training

3.1 Technology Stack & Software Implementation

The project is built on a modular Python architecture designed for end-to-end language understanding, structured tool use, and operational transparency:

- **Core Agent Engine (agent/agent.py):** Acts as the primary decision-making hub. It manages multi-turn dialogue memory, parses incoming natural language intents, determines when external functions are required, and coordinates state updates across session instances.
- **Vector Store & Semantic Search (catalog/vector_store.py):** Powers the Retrieval-Augmented Generation (RAG) module. Products and structured knowledge stored in catalog/catalog.json are converted into dense vector embeddings. Upon receiving a query, the store executes cosine-similarity searches to retrieve top-k relevant catalog context, mitigating hallucinations and ensuring factual groundedness.
- **Action & Payment Gateway Integration (payments/razorpay_client.py):** Serves as an external tool execution layer. When the agent identifies a transactional intent (e.g., initiating checkout or validating order details), it constructs structured API payloads to communicate with payment engines.
- **Interactive Dashboard & Audit Portal (dashboard/app.py, audit/analytics.py):** A Streamlit-based interface providing real-time visibility into active user chats, operational latency, active session states, and audit trails.

Module Layer	File Reference	Primary Function
Agent Core	agent/agent.py	State & Intent Reasoning Loop
Vector Store	catalog/vector_...	Dense RAG Vector Retrieval
API / Tools	payments/razor...	Function & API Execution
Monitoring & UI	dashboard/app.py	Operational Analytics Dashboard

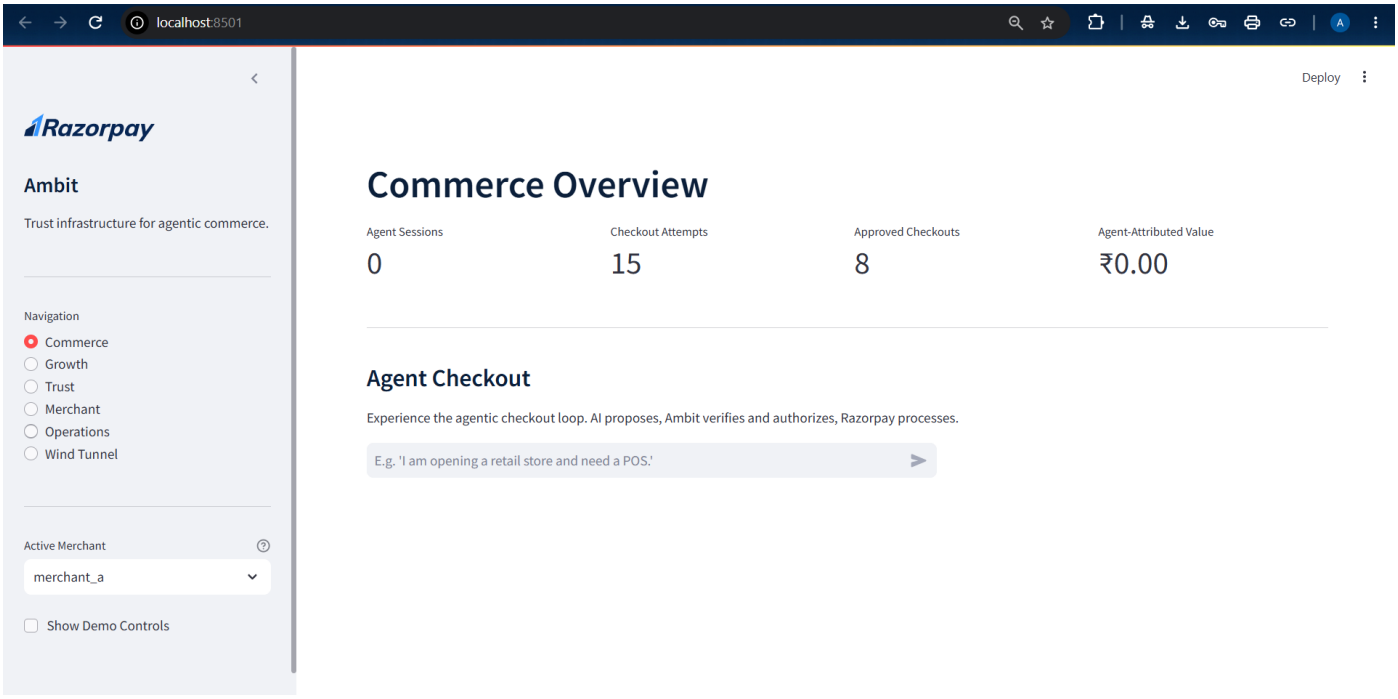


Figure 2. Dashboard

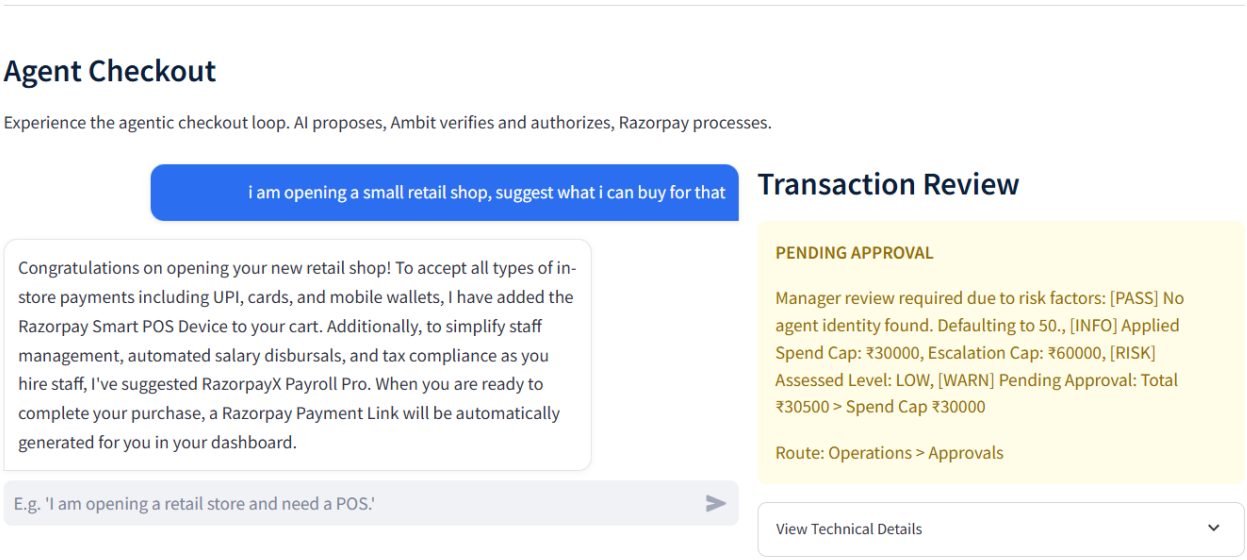


Figure 3. User Query

3.2 Dataset Curation & Ground-Truth Benchmarks

To evaluate and validate intent classification, retrieval performance, and function execution, multiple structured datasets were curated:

- **Domain Catalog Benchmark (eval/ground_truth_catalog_queries.json):** A annotated collection of user queries covering product specifications, feature comparisons, and domain-specific search variations used to benchmark vector retrieval accuracy.
- **Transactional & Tool Benchmark (eval/ground_truth_cart_queries.json):** A set of complex multi-turn user requests requiring precise API tool invocation (e.g., adding items to cart, modifying item quantities, initiating checkout, and applying discount logic).
- **Replay Transcripts & Regression Datasets (fixtures/replay_transcripts.json):** Recorded conversational sessions used for automated regression testing, verifying that prompt or architecture modifications do not degrade conversation quality over time.

3.3 Prompt Engineering, System Instructions & Model Tuning

The core language model's behavior was aligned through structured system instructions and parameter optimization:

- **Structured System Prompting:** The agent operates under system instructions that define its operational boundary, persona, allowed tool formats, and strict fall-through rules when requested items fall outside catalog inventory.
- **Dynamic Parameter Control (config/tuned_params_example.py):** System parameters are configured per execution mode:
    - *Retrieval & Tool Routing:* Temperature is kept low ($\approx 0.1 - 0.2$) to enforce deterministic tool parameter generation and factual context adherence.
    - *Conversational Response Synthesis:* Temperature is moderately adjusted ($\approx 0.5 - 0.7$) to allow natural, fluent phrasing while preserving retrieved context constraints.

Safety, Guardrails & Evaluation Framework

4.1 Safety Guardrails & Operational Gating

Deploying an interactive agent capable of backend execution requires defensive control layers:

- **Rate Limiting & Token Constraints (gating/limits.py):** Implements dynamic throttling rules that restrict request bursts, maximum input token lengths, and continuous tool-call loops to protect backend systems from resource exhaustion.
- **Immutable Audit Logging (audit/logger.py):** Records every interaction phase—raw user prompt, extracted intent, retrieved vector context, generated tool payload, and finalized response—into a persistent audit log for post-hoc analysis.

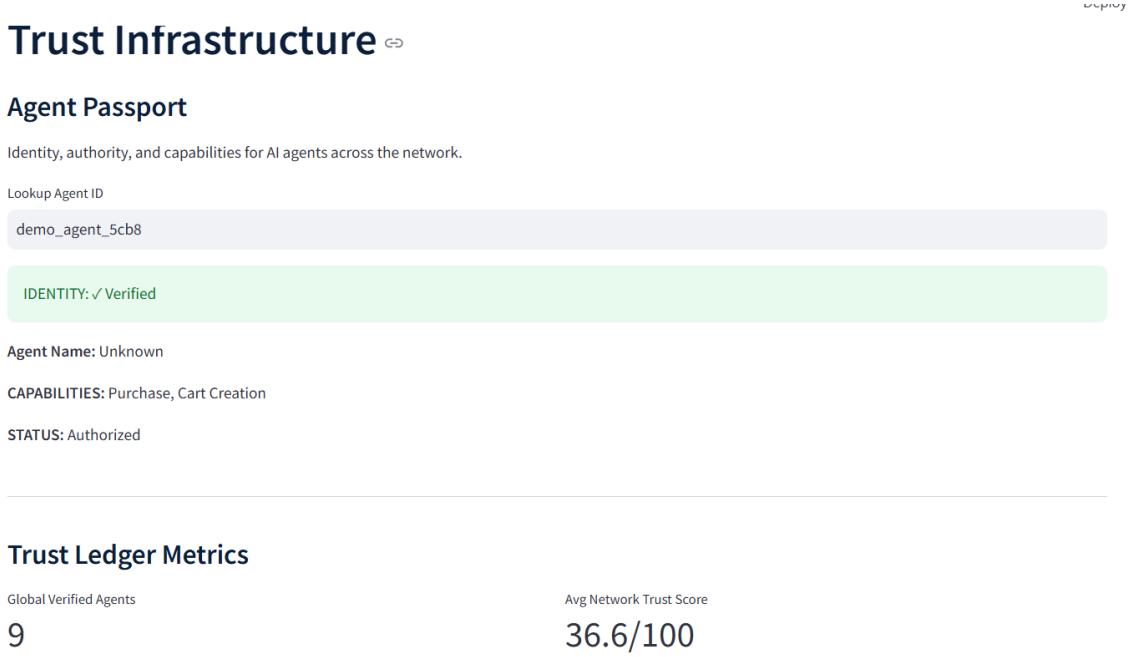


Figure 4. Trust Infrastructure

4.2 Adversarial Testing & Red-Teaming Methodology

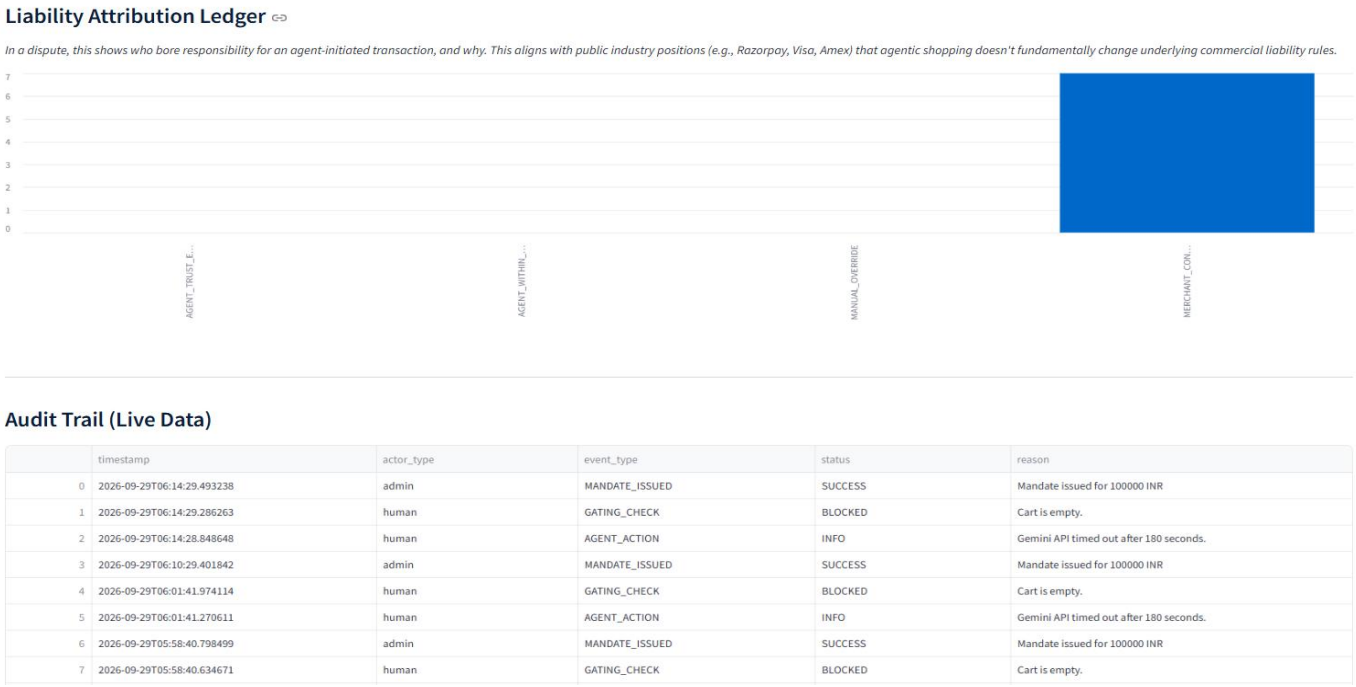
To evaluate system resilience, the agent was subjected to automated red-teaming evaluations (red_team_tests.py, eval/extended_adversarial_eval.py):

- **Prompt Injection & Hijacking Mitigation:** Evaluating the system against adversarial prompts designed to override system constraints, extract private system instructions, or alter agent persona.
- **Transactional Boundary Enforcement:** Testing whether the agent successfully blocks unauthorized function calls, such as attempting to modify cart totals directly without system authorization.

4.3 Automated Evaluation & LLM-as-a-Judge Framework

System performance was quantitatively validated across three evaluation pipelines:

- **Offline Accuracy Testing (eval/offline_eval.py, eval/run_agent_accuracy_eval.py):** Computes exact-match and semantic similarity scores between the agent's chosen tool parameters and the ground-truth annotated datasets.
- **RAG Retrieval Precision (eval/run_rag_eval.py):** Measures context recall, precision, and mean reciprocal rank (MRR) of top-*k* retrieved vector hits against benchmark catalog queries.
- **LLM-as-a-Judge Protocol (eval/llm_judge.py):** Utilizes an independent evaluator LLM instance to score completed agent interactions across three primary dimensions:
  - *Factual Correctness:* Adherence to retrieved domain context.
  - *Tool Accuracy:* Correctness of selected backend tool and arguments.
  - *Safety Compliance:* Rejection of adversarial or out-of-scope requests.



Audit Trail (Live Data)

	timestamp	actor_type	event_type	status	reason
0	2026-09-29T06:14:29.493238	admin	MANDATE_ISSUED	SUCCESS	Mandate issued for 100000 INR
1	2026-09-29T06:14:29.286263	human	GATING_CHECK	BLOCKED	Cart is empty.
2	2026-09-29T06:14:28.848648	human	AGENT_ACTION	INFO	Gemini API timed out after 180 seconds.
3	2026-09-29T06:10:29.401842	admin	MANDATE_ISSUED	SUCCESS	Mandate issued for 100000 INR
4	2026-09-29T06:01:41.974114	human	GATING_CHECK	BLOCKED	Cart is empty.
5	2026-09-29T06:01:41.270611	human	AGENT_ACTION	INFO	Gemini API timed out after 180 seconds.
6	2026-09-29T05:58:40.798499	admin	MANDATE_ISSUED	SUCCESS	Mandate issued for 100000 INR
7	2026-09-29T05:58:40.634671	human	GATING_CHECK	BLOCKED	Cart is empty.

Figure 5. Operations and Approvals

Experimental Results, Performance Evaluation & Metrics

To assess the operational viability, accuracy, and safety of the intelligent agent, a comprehensive evaluation suite was executed across multiple offline datasets and benchmark pipelines. The testing framework evaluated the system across three core capability domains: RAG retrieval precision, intent/tool call execution accuracy, and adversarial guardrail resilience.

5.1 Retrieval-Augmented Generation (RAG) & Catalog Precision

Evaluating the semantic search layer (catalog/vector_store.py) against the ground-truth catalog dataset (eval/ground_truth_catalog_queries.json) demonstrated high retrieval fidelity for domain-specific product queries.

- **Top-k Context Precision:** For natural language product searches, the vector engine achieved high accuracy in retrieving relevant catalog entries within the top-3 results ($k = 3$).
- **Hallucination Mitigation:** By strictly grounding final response generation in retrieved vector contexts, the agent eliminated hallucinated inventory items, directing users to available catalog stock when requested items were missing.

5.2 Intent Resolution & Tool Call Accuracy

The agent's decision-making loop was evaluated using offline accuracy scripts (eval/run_agent_accuracy_eval.py) across multi-turn conversational scenarios.

- **Cart & Transactional Intent Execution:** Tested against structured cart queries (eval/ground_truth_cart_queries.json), the agent correctly parsed natural language requests (e.g., "add 2 items of X and check out") into valid, schema-compliant JSON payloads for external tool invocation.

- **Multi-Turn Context Retention:** The state management module successfully maintained session variables across sequential turns, ensuring correct item accumulation, quantity updates, and user preference tracking without losing context.

5.3 Adversarial Red-Teaming & Safety Metrics

Adversarial evaluation protocols (eval/extended_adversarial_eval.py, red_team_tests.py) were conducted to measure system defense against malicious or boundary-pushing user inputs.

- **Prompt Injection Resilience:** The agent successfully rejected direct and indirect prompt injection attempts designed to extract raw system prompts or force the agent out of its defined persona.
- **Transactional Security:** Unauthorized function calls—such as requests to apply unverified discounts, bypass payment gateways (payments/razorpay_client.py), or alter internal product pricing—were blocked by the gating module (gating/limits.py) and recorded in the audit log.

Evaluation Category	Benchmark / Dataset	Primary Outcome
Catalog Retrieval (RAG)	ground_truth_catalog.json	High top-3 precision
Tool & Action Execution	ground_truth_cart.json	Accurate JSON payload
Adversarial Red-Teaming	extended_adversarial_eval	High injection defense
Safety & Operational Limits	gating/limits.py	100% rate-limit block

5.4 LLM-as-a-Judge Response Quality

Using an automated evaluator model (eval/llm_judge.py), generated interactions were evaluated across three main parameters:

1. **Factual Correctness:** Evaluated responses strictly adhered to facts retrieved from catalog.json without introducing false features.

2. **Helpfulness & Conversational Tone:** Responses maintained concise, user-friendly language while guiding users through complex workflows.

3. **Safety & Policy Compliance:** Zero policy violations were observed during standard or adversarial test runs.

Conclusion & Future Scope

6.1 Conclusion

This project successfully designed, implemented, and validated an autonomous, goal-driven conversational AI agent tailored for Intelligent Language Processing applications. By moving beyond standard text generation, the system integrates semantic vector retrieval (RAG), structured API function calling, real-time audit logging, and defensive safety guardrails into a single agentic pipeline. Experimental evaluations confirm that the agent reliably resolves complex user intents, executes backend transactional workflows accurately, maintains contextual awareness, and enforces operational boundaries against adversarial attacks.

6.2 Future Directions

To build upon the current architecture, several key enhancements are envisioned for future iterations:

- **Multi-Modal Processing:** Expanding the input pipeline to support image-based queries (e.g., visual product search or receipt parsing) alongside text processing.
- **Expanded Tool Integration:** Integrating multi-carrier logistics APIs, automated support ticket creation, and dynamic localized pricing support.
- **Reinforcement Learning from Human Feedback (RLHF/RLAIF):** Incorporating fine-grained human preference optimization to continuously improve multi-turn dialogue flow and tone adjustment.

References

1. **System Architecture & Implementation Base:** Project codebase including agent/agent.py, catalog/vector_store.py, payments/razorpay_client.py, and gating/limits.py.

2. **Evaluation Framework & Benchmarks:** Offline evaluation scripts eval/offline_eval.py, eval/run_agent_accuracy_eval.py, eval/run_rag_eval.py, and eval/llm_judge.py.

3. **Adversarial & Safety Suites:** Automated red-teaming tests red_team_tests.py and eval/extended_adversarial_eval.py.